Proceedings of the Fourth International Conference on Ubiquitous Computing and Intelligent Information Systems (ICUIS-2024)
IEEE Xplore Part Number: CFP24UG3-ART; ISBN: 979-8-3315-2963-5

Implementation of 8-bit SPI Protocol using System Verilog and UVM

M. Saravanan[1],
Department of ECE,
Sri Eshwar College of Engineering,
Coimbatore, Tamilnadu, India
saranecedgl@gmail.com.

R. Sandhiya[2],
Department of ECE,
Sri Eshwar College of Engineering,
Coimbatore, Tamilnadu, India
santhiyar71@gmail.com

S. Sivaranjani[3],
Department of ECE,
Sri Eshwar College of Engineering,
Coimbatore, Tamilnadu, India
sivaranjanisivakumar114@gmail.com.

K. Tamilarasi[4],
Department of ECE,
Sri Eshwar College of Engineering,
Coimbatore, Tamilnadu, India
tamilarasikannan928@gmail.com

M.Tamilarasan[5],
Department of ECE,
Sri Eshwar College of Engineering,
Coimbatore, Tamilnadu, India
tamil882005@gmail.com

G.M. Varshini[6],
Department of ECE,
Sri Eshwar College of Engineering
Coimbatore, Tamilnadu, India
Varshinigm2004@gmail.com

2024 4th International Conference on Ubiquitous Computing and Intelligent Information Systems (ICUIS) | 979-8-3315-2963-5/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICUIS64676.2024.10867233

Abstract: Currently, the design and verification of efficient communication protocols are essential in embedded systems. This paper focuses on the development and verification of an 8-bit Serial Peripheral Interface (SPI) protocol using System Verilog and Universal Verification Methodology (UVM). SPI, a popular protocol in embedded communication, enables high-speed, synchronous, full-duplex data transfer between a master and one or more slave devices. By utilizing SystemVerilog for hardware modeling and UVM for a modular, reusable testbench framework, this approach enhances the verification process's efficiency and accuracy. Various operational scenarios for the SPI protocol are tested, ensuring robust performance in communication systems.

Keywords- SPI Protocol, SystemVerilog, Universal Verification Methodology (UVM), VLSI Design, Embedded Systems, Communication Protocols, Modular Testbench

I. INTRODUCTION

System-on-chip designs typically include a wide range of peripherals, such as analog-to-digital converters (ADCs), registers, memory units, and digital-to-analog converters (DACs). As a result, efficient data transmission and reception between these connected peripherals are essential for SoC functionality. The Serial Peripheral Interface (SPI) is one of the most widely used serial protocols for both inter-chip and intra-chip low-to-medium speed data transfers. SPI facilitates communication between processors and devices like external EEPROMs, DACs, ADCs, and similar components. In the landscape of communication protocols, SPI is often referred to as a "small" protocol. It's crucial to recognize the unique role of each protocol; for instance, Ethernet, USB, and SATA are designed for "outside-the-box communications," supporting data exchange between entire systems. In contrast, SPI is specifically designed for interconnecting integrated circuits for low-to-medium speed data transfer on-board with peripheral devices [1 – 3]. Data exchange in SPI occurs between a master device and one or more slave devices. When data is transmitted by a device, incoming data must be read before the next transmission can occur, ensuring that data exchange is continuous and synchronized [4]. In SPI, each device must be capable of both transmitting and receiving data. The master device governs the clock line (SCK), controlling data exchange between devices through this clock line. Serial Peripheral Interface (SPI) is a commonly used critical communication protocol in a variety of embedded systems. It also enables high-speed synchronous data transfer between devices. The main objective is to enable efficient, dependable communication, especially in systems where pin count is critical. This study focuses on developing an 8-bit SPI protocol with System Verilog, a strong hardware description and verification language In scenarios such as processor-to-processor and processor-to-peripheral communication, data integrity and speed are critical.

The methodology for implementing and verifying the SPI protocol is based on two key approaches: System Verilog for coding the protocol and Universal Verification Methodology (UVM) for verification. System Verilog allows for precise hardware modeling of the SPI protocol, and UVM provides a standardized testbench framework for comprehensive and systematic verification of the communication system. Using these protocols guarantees that the design is robust and performs properly in a variety of scenarios. UVM also offers a verification environment that is reusable and scalable which results in improving verification efficiency and simplifies the testing procedure. Nonetheless, the SPI protocol's lack of a flow control mechanism and addressing scheme restricts its use in increasingly complex systems. In addition, the slave devices have to obey the master's commands to the letter, and in multi-slave deployments, it can be challenging to manage several devices due to the absence of a formal addressing structure [5 – 7]. While System Verilog offers a versatile and powerful solution to implementing the 8-bit SPI protocol, UVM verification

979-8-3315-2963-5/24/$31.00 ©2024 IEEE

allows for scalability and reuse. Method 1, which is direct implementation with System Verilog, has the advantage of having more control over hardware modeling, but it may lack systematic testing. Method 2, which uses UVM, addresses this issue by providing a reusable, modular verification environment, which necessitates additional learning and setup time [8]. By comparing various methods, we hope to demonstrate how combining them can overcome limits such as debugging complexity and increase overall system verification efficiency [9 -11]. The novelty of the project is unlike previous works that primarily focused on SystemVerilog and UVM, our paper uniquely integrates both methodologies to deliver a comprehensive and robust verification framework.

II. SPI PROTOCOL

The Serial Peripheral Interface (SPI) is a widely used communication protocol, offering synchronous, full-duplex data transmission at high speeds over short distances—typically between devices on the same circuit board. As shown in the figure 1, SPI is commonly employed to enable communication between microcontrollers or microprocessors and various peripherals [12]. To ensure synchronized data transmission and reception, SPI requires a clock signal. Operating in a master-slave configuration, SPI supports both single master-single slave and single master-multiple slave architectures. Communication between master and slave devices occurs over a synchronous clock signal generated by the master. Only four signal lines are needed for communication between master and slave devices in SPI [14 -15].

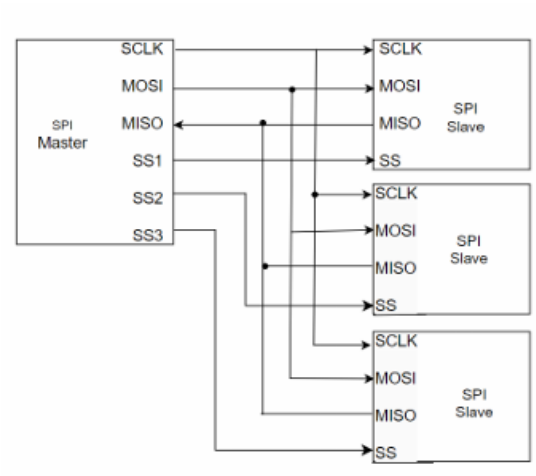

Fig. 1. SPI Protocol

SPI consists of four primary signals: Serial Clock (SCK) This clock signal synchronizes data transmission between the master and slave over the MOSI and MISO lines. An entire byte of data is exchanged between master and slave within eight clock cycles, with SCK generated by the master and serving as the input clock for the slave. Chip Select (CS) / Slave Select (SS): This line selects the slave device for communication. It remains active-low during data transfer [6-8]. Master In Slave Out (MISO): Through the MISO line, the master receives data from the slave. Data is transmitted serially, with the most significant bit (MSB) sent first. If the slave is not selected, the MISO line is held in a high-impedance state. Master Out Slave In (MOSI): Through the MOSI line, the master transmits data to the slave in a serial, one-way direction, also starting with the MSB. In SPI, the rising or falling clock edges dictate when data is sent and received by the master and slave, respectively. SPI Communication setup where a single SPI Master device is connected to multiple SPI Slave devices (four in this case), allowing the master to communicate with each slave individually over a shared bus. The SPI protocol utilizes four main signal lines: SCLK (Serial Clock), MOSI (Master Out Slave In), MISO (Master In Slave Out), and SS (Slave Select). The SCLK line, generated by the master, is a clock signal that coordinates the timing of data transmission between the master and all slaves. This clock signal is shared among all slave devices to ensure synchronized data transfer. The MOSI line is used by the master to send data to the slaves, allowing it to transmit information in a serial format. Each slave listens on this line, but only the selected slave processes the data being sent. Conversely, the MISO line is used by the active slave to send data back to the master, creating a two-way communication link. To avoid conflicts when multiple slaves are connected to the same MISO line, only the select slave drives the line, while the others remain in a high-impedance (tri-state) state, effectively disconnecting them from the line to prevent data overlap or interference. When the master pulls one of these SS lines low, it enables the corresponding slave to communicate, while the other slaves remain inactive. For example, if the master wants to communicate with the first slave, it pulls SS1 low, activating only that slave and keeping others in standby mode. This selective activation enables the master to communicate with multiple slaves over shared MOSI and MISO lines without causing data conflicts. Once the slave is selected, the master initiates data transfer over a series of clock pulses.

In a typical 16-bit SPI setup, the master and active slave exchange 16 bits of data in a full-duplex manner, meaning data is sent on the MOSI line from the master to the slave while, at the same time, data from the slave is sent to the master over the MISO line. The SS (Slave Select) lines play a crucial role in this configuration, as they allow the master to select and activate a specific slave for communication. Each slave has its dedicated SS line, which is controlled directly by the master. When the master pulls one of these SS lines low, it enables the corresponding slave to communicate, while the other slaves remain inactive. For example, if the master wants to communicate with the first slave, it pulls SS1 low, activating only that slave and keeping others in standby mode. Each slave module would also have logic to drive the MISO line with its data output when active. The use of shift registers in both the master and slave modules is a common approach to handling serial data transfer, ensuring the correct timing and format for 8-bit data frames.

III. PROPOSED ARCHITECTURE

(a) System Verilog Architecture

The System Verilog testbench architecture provides a structured, modular approach to verifying a Design Under Test (DUT), as shown in the figure 2. At the highest level is the Top Module, which instantiates the

979-8-3315-2963-5/24/$31.00 ©2024 IEEE

DUT and the Environment, setting up the necessary connections for simulation. The Environment serves as a container for essential verification components, organizing and grouping them to ensure modularity and reusability. Within the Environment, the Agent contains several critical sub-components, including the Generator, BFM (Bus Functional Model), Monitor, and Coverage components. The Generator creates stimulus or test patterns, which are often randomized or constrained to simulate a wide range of operating scenarios and passes these patterns to the BFM.

Acting as an interface between the testbench and the DUT, the BFM converts the high-level transactions from the Generator into pin-level signals that drive the DUT inputs. Meanwhile, the Monitor passively observes the communication between the BFM and the DUT, capturing output data without interfering with the simulation. This data is then sent to the Scoreboard, which compares it with expected outputs to verify the DUT's functionality, checking for any mismatches and logging errors.

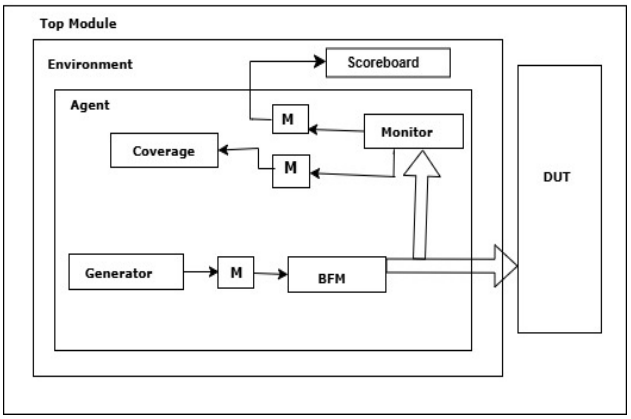

Fig. 2 System Verilog Architecture

Additionally, the Coverage component records functional coverage data based on the monitored transactions, tracking how much of the DUT's functionality has been exercised, which ensures that all required scenarios have been tested. The architecture also includes Mailboxes (shown as "M" in the diagram) that serve as transaction queues, facilitating asynchronous communication between components like the Generator, BFM, and Monitor, allowing for efficient data transfer and coordination. This modular architecture, with each component performing a specific role, enhances testbench flexibility, reusability, and scalability, making it well-suited for rigorous verification of complex designs and adaptable for different DUTs or testing scenarios.

The System Verilog testbench architecture leverages several advanced features to enhance the verification process and increase test coverage. One of these features is Functional Coverage, which provides detailed insights into which parts of the DUT have been tested. By setting specific coverage goals, the verification team can identify any untested scenarios and adjust the stimulus to achieve comprehensive coverage, ensuring that every critical path in the DUT is exercised. Another powerful aspect of this architecture is the use of Assertions, which are embedded within the testbench to check for expected behavior in real time. Assertions act as checkpoints during simulation, quickly identifying protocol violations or incorrect behavior by verifying that signals follow specified timing, order, or state transitions. This immediate feedback accelerates debugging and enhances the accuracy of test results. System Verilog also supports Randomization and Constraints in stimulus generation, enabling the creation of complex, varied test scenarios that help expose corner cases that may not be covered by manual test patterns. Randomization introduces a degree of unpredictability in test patterns, while constraints guide this randomness to ensure valid transactions within the DUT's expected operating parameters. Additionally, Virtual Interfaces are used in the testbench to simplify connections between components and increase modularity. Virtual interfaces allow different testbench components to access shared signals dynamically, making it easier to swap modules or reconfigure the testbench for different DUT configurations.

(b) UVM Architecture

In IC design, the Universal Verification Methodology (UVM) is pivotal for establishing a robust verification framework that confirms a module meets its specifications. Originating from the Open Verification Methodology (OVM) and leveraging System Verilog, UVM emphasizes key features like code reusability, environment construction, random stimulus generation, and rigorous Design Under Test (DUT) verification to meet project requirements. At its core, UVM organizes its classes into three main categories: uvm_object, the base class for data and operational methods, from which essential classes like uvm_component and uvm_transaction inherit; uvm_transaction, which is central to generating stimuli; and uvm_component, a static class throughout simulation that encompasses primary components such as the Top, Agent, Sequence Item, Sequence, Test, Environment, Driver, Sequencer, Monitor, and Scoreboard.

979-8-3315-2963-5/24/$31.00 ©2024 IEEE

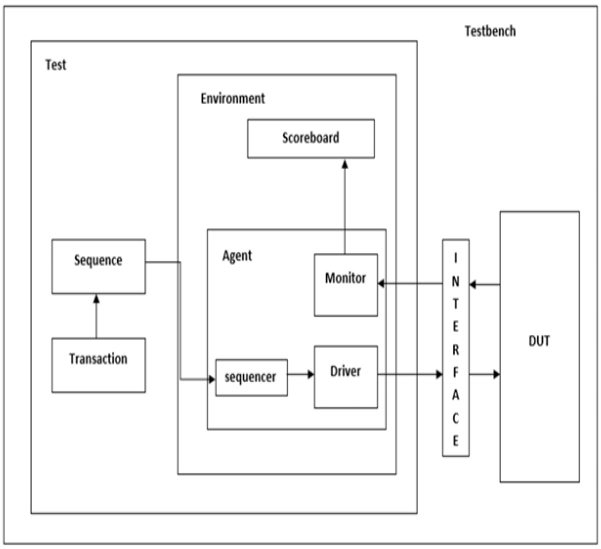

Fig. 3 UVM Architecture

Each UVM component functions through defined simulation phases that organize and control the verification process. The Build Phase is responsible for creating and configuring component hierarchies according to configuration and factory settings, while the Connect Phase links ports among various UVM components. The Run Phase handles runtime simulation tasks, and the Extract Phase gathers data from scoreboards and monitors for functional coverage analysis. Following this, the Check Phase verifies DUT behavior, identifying errors in the testbench execution, and the Report Phase displays simulation results while saving them for analysis. The Final Phase then signals the end of the simulation, ensuring all components conclude safely.

As shown in the figure 3, the UVM testbench architecture consists of several interconnected components: the Testbench, which instantiates the DUT and test classes and establishes their connections; the Test, which configures the testbench and initiates stimulus sequences; and the Environment, which groups reusable verification components, such as the scoreboard and agent, into modular units. Within the Environment, the Agent includes the Monitor, Driver, and Sequencer, working as a signal entity through the TLM interface. The Sequencer generates and manages transactions, directing them to the Driver, while the Driver sends data to the DUT and reference model via the interface. The Monitor samples data from the DUT, recording and comparing transaction data, and the Scoreboard verifies the DUT's functionality through embedded checkers.

Additional aspects further enhance UVM's adaptability and effectiveness. For instance, UVM employs a Factory Pattern for dynamic object creation and substitution, allowing flexible testbench configurations without code modifications. The Configuration Database centralizes parameter passing across the testbench, ensuring consistency and simplifying adjustments. Transaction-Level Modeling (TLM) interfaces abstract data transactions, decoupling components for modular design and scalability. Sequences and Virtual Sequences allow for synchronized stimulus generation across multiple agents, enabling coordinated, complex scenarios. Phase Jumping provides control over simulation flow by allowing transitions between phases based on test requirements. Together, these features make UVM an industry-standard approach for creating scalable, reusable, and adaptable verification environments.

IV. RESULTS AND DISCUSSION

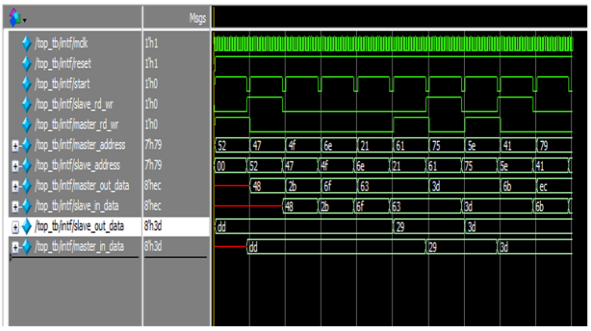

Fig.4. Simulated Waveform of SPI

Figure 4 shows the simulated waveform of the serial peripheral interface data exchange between the master and slave during the MISO and MOSI modes of operation. This also shows data transfer and control signals over time, where various signals are toggling in response to specific clock cycles, indicating data flow and synchronization.

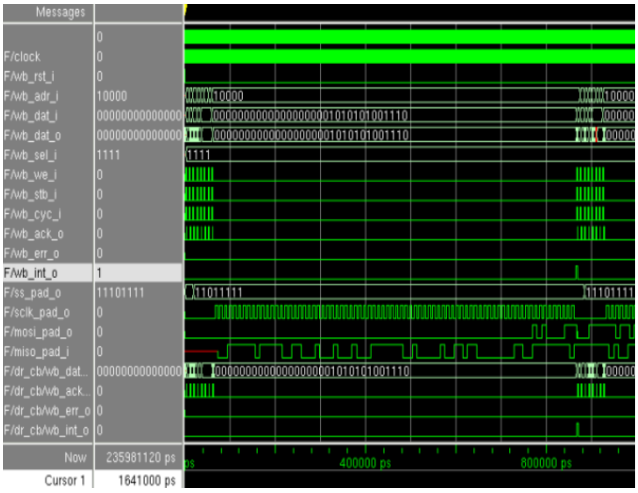

Fig. 5. SPI Master Slave Core

Figure 5 depicts the simulated waveform of the serial peripheral interface data exchange between the master and slave core. The timing diagram shows transitions and data changes across different signals, illustrating the behavior of the circuit over time.

979-8-3315-2963-5/24/$31.00 ©2024 IEEE

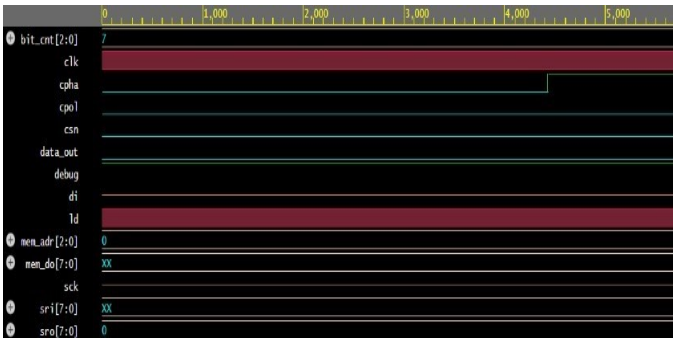

Fig. 6. SPI Slave Waveform

Figure 6 illustrates the slave functioning waveform. The figure shows the behavior of these slave signals over time or control sequence.

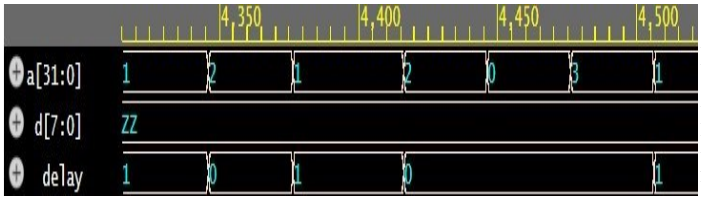

Fig. 7. Read Operation of the SPI Master

Figure 7 shows the read operation of the SPI master. The delay signal demonstrates timing delays for synchronization during the read process.

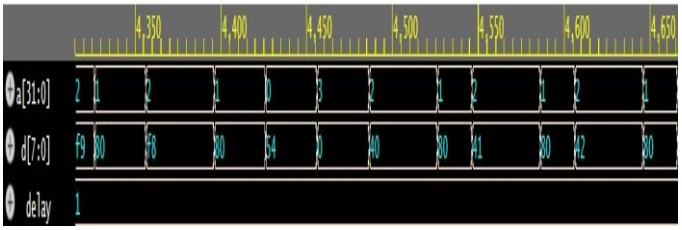

Fig. 8. Write Operation of SPI Master

Figure 8 demonstrates the write operation of the SPI Master. The delay signal indicates the timing intervals used for synchronization, ensuring data stability during each transfer.

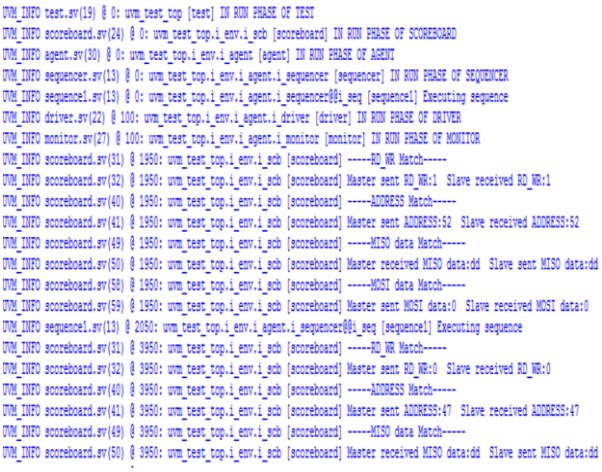

Fig. 9 UVM Report of SPI's Data Transaction Between Master and Slave

Figure 10 gives the successful data transfer between master and slave and vice-versa. Transaction details like RD_WR Match, address Match, MISO data Match and MOSI data Match are shown on successful data exchanged between the master and slave

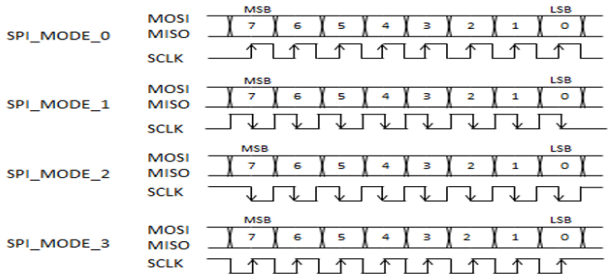

Fig. 10 Modes of Operation Simulation

Figure 10 shows the different modes of operation of the SPI master and slave based on the clock polarity and clock phase. The four SPI modes (0–3) define the clock polarity and phase for data sampling and shifting. Modes 0 and 1 have a low idle clock, while modes 2 and 3 have a high idle clock; modes 0 and 2 sample on the rising edge, and modes 1 and 3 sample on the falling edge.

Table 1 Modes of operation

Clock Phase	Clock polarity	Modes of operation
0	0	Mode 0
0	1	Mode 1
1	0	Mode 2
1	1	Mode 3

Table 2 Code Coverage of Master Module

Total Coverage:					70.00%	87.15%
Coverage Type	Bins	Hits	Misses	Weight	% Hit	Coverage
Statements	18	18	0	1	100.00%	100.00%
Branches	22	21	1	1	95.45%	95.45%
FEC Conditions	10	9	1	1	90.00%	90.00%
Toggles	190	120	70	1	63.15%	63.15%

979-8-3315-2963-5/24/$31.00 ©2024 IEEE

Table 3 Code Coverage of Slave Module

Total Coverage:					63.81%	79.70%
Coverage Type	Bins	Hits	Misses	Weight	% Hit	Coverage
Statements	14	14	0	1	100.00%	100.00%
Branches	10	10	0	1	100.00%	100.00%
FEC Conditions	5	3	2	1	60.00%	60.00%
Toggles	170	100	70	1	58.82%	58.82%

Table 1 shows the mode of operation and Table 2 illustrates the code coverage details of the SPI master functional module. Table 3 shows the code coverage details of the SPI slave functional module. The SPI protocol operates in four distinct modes, determined by the Clock Phase (CPHA) and Clock Polarity (CPOL) settings. In Mode 0 (CPHA = 0, CPOL = 0), the clock polarity is low, and data is sampled on the rising edge and shifted out on the falling edge. Mode 1 (CPHA = 1, CPOL = 0) also has a low clock polarity, but data is sampled on the falling edge and shifted out on the rising edge. Mode 2 (CPHA = 0, CPOL = 1) sets the clock polarity to high, where data is sampled on the rising edge and shifted out on the falling edge. Finally, Mode 3 (CPHA = 1, CPOL = 1) also has a high clock polarity, with data being sampled on the falling edge and shifted out on the rising edge. Each mode offers different timing for data sampling and shifting based on the phase and polarity configuration.

V. CONCLUSION

This research study successfully demonstrates the design and verification of an 8-bit SPI protocol using SystemVerilog and UVM, providing a scalable and reusable testbench architecture suitable for modern VLSI designs. By achieving efficient data exchange between master and slave devices, validated through comprehensive simulation, this work highlights the importance of robust communication protocols in VLSI circuits where high integration and reliable interconnectivity are crucial. Future work could explore implementing more complex SPI systems, such as those supporting higher bit widths and multiple slaves in larger VLSI systems, or integrating error detection and correction mechanisms to enhance reliability. Additionally, multi-master configurations and clock synchronization across VLSI blocks could be addressed to broaden the protocol's applications. Expanding the protocol's usability for IoT, automotive, and AI-driven applications can further support the development of versatile VLSI solutions adaptable to evolving industry demands.

REFERENCES

[1] Polsani Pallavi, V. Priyanka Brahmaiah,Y. Padma Sai," Design & verification of SPI peripheral interface(SPI) Protocol", Vol. 8, Issue 6, 2020

[2] S.Choudhury, G.K.Singh, R.M.Mehra, "Design & verification serial Peripheral Interface(SPI) Protocol for Low Power Applications", Vol. 3, Issue 10, 2014

[3] P.R. Reddy, P. Sreekanth and K.A. Kumar, "Serial peripheral interface-master universal verification component using UVM", International Journal of Advanced Technologies in Engineering and Management Sciences, Vol. 3, 2017.

[4] Veda Patil, Vijay Dahake, Dharmesh Verma, Elton Pinto, "Implementation of SPI Protocol in FPGA" International Journal Of Computational Engineering Research, Vol. 3, 2013

[5] M.Sandyal, K.Rajasekhar, "Design and Verification of Serial Peripheral Interface", Vol. 3, Issue. 4, 2012

[6]K.Aditya, M.Sivakumar, Fazal Noorbasha,T.Praveen Blessington " Design and Functional Verification of a SPI Master Slave Core Using System Verilog", Vol. 2, Issue. 2, 2012

[7] K.V. Ashok Kumar, M. Santhosh Krishna," Design and Functional Verification of a SPI Master Slave Core using UVM", Vol. 04, Issue. 51, 2015

[8] Rahul Jandyam, Sanjay Reddy Kandi, Umar Farooq Mohammad," Design and Implementation of Spi Module in Verilog HDL using FPGA design Flow", Vol. 5, Issue. 4, 2017

[9] K. Aditya, M. Sivakumar, Fazal Noorbasha, T.PraveenBlessington, "Design and Functional Verification of a SPI Master Slave Core Using System Verilog,", Vol. 2, 2012.

[10] Abhijeet Kumar," Implementation of Low Power SPi Protocol with Clock Domain Crossing, Vol. 1, Issue. 2, 2014.

[11] Aman Kulkarni, S.M. Sakthivel," UVM methodology based functional verification of SPI protocol",2020.1-3, 1 Aug 2016.

[12] Anuradha, M. Saravanan, " Design and Development of a Portable ECG Acquisition System" International Journal of Advanced Research in Computer and Communication Engineering, Vol. 5, Issue 2, 2016

[13] P.M. Aarthi; A. Dinesh; T. Janani; V.Ananth Kumar; M. Saravanan; J. Ajayan, "Study of Performance of High-Speed Low-Power Differential Input Based Dynamic Comparator Using 22 nm CMOS Technology" DOI: 10.1109/ICACCS48705.2020.9074441, April 2020

[14] G Ramesh, P Manikandan, P Naveen, M Saravanan, SR Ashok Kumar, C Swedheetha, "Energy efficient high-performance adder/subtractor circuits" 2022 3rd International Conference on Smart Electronics and Communication (ICOSEC), pp 333-338, 2022

[15] M Saravanan, KA Balasuriya, D Deepak Raj, B Dharun, R Dinesh Kumar, Palanichamy Naveen, " Design and Implementation of 32-bit Signed Divider for VLSI Applications", 2023 7th International Conference on Electronics, Communication and Aerospace Technology (ICECA), pp 316-320, 2023

979-8-3315-2963-5/24/$31.00 ©2024 IEEE